SFC-007 — Implications for SCR and AI Agents
Commitment Governance, Gap Bridging, and Runtime Extension Intelligence

Sizhe Tan
with chatGPT (OpenAI)
2026-06-25

Repository: https://github.com/sizhet/Structural-Feasibility-Confidence-SFC

1. Introduction

Structural Feasibility Confidence (SFC) is not only a theory about human craftsmanship, engineering estimation, or task planning in the abstract.
It has direct implications for **Structural Cognitive Runtime (SCR)** and for future AI agents that must operate under partial novelty.
If SCR is concerned with how structural cognitive systems maintain assets, detect gaps, update runtime state, and generate behavior, then SFC adds a critical missing layer:
the layer that decides how far the system should trust its current structures when facing a target that is not yet fully confirmed.
This is the layer of:

- commitment governance,
- extension judgment,
- gap-bridging courage and restraint,
- prototype-versus-promise decisions,
- and boundary-aware planning.

Without something like SFC, an AI runtime is pushed toward two bad extremes:

1. **Rigid replay conservatism**
   It only trusts already executed paths and cannot plan meaningfully beyond them.

2. **Hallucinated overreach**
   It invents unsupported commitments because it lacks a disciplined notion of feasibility support.

SFC is intended to provide the middle layer that avoids both.

2. Why SCR Needs an SFC Layer

SCR already deals with important runtime questions:

- What structural assets exist?
- What gaps have been detected?
- Which triggers are active?
- What updates should be made to the runtime state?
- Which task or action path should be executed next?

But once the system begins to operate beyond pure replay, a further question becomes unavoidable:
How should the runtime judge whether a not-yet-confirmed target is still within supported reach?
That question appears whenever the runtime must:

- bridge a gap,
- attempt a new task variant,
- reuse a known path in a modified setting,
- commit to a Task Calling Graph not yet fully backed by action traces,
- or decide whether to answer, refuse, defer, or escalate.

This is exactly where SFC belongs.

3. SFC as a Runtime Middle Layer

A useful way to place SFC in runtime architecture is to treat it as a middle layer between structural detection and structural commitment.
A simplified picture is:

1. **Structural Observation / Asset Layer**
   What graphs, modules, precedents, and runtime resources are currently available?

2. **Gap / Mismatch / Goal Layer**
   What is missing relative to the target?

3. **SFC Layer**
   Given the target and the currently available support, how feasible is extension, and what kind of commitment is justified?

4. **Execution / Bridging / Escalation Layer**
   Try, stage, defer, ask for help, narrow scope, or refuse.

This placement matters because it prevents gap detection from automatically implying gap bridging.
A gap must first be judged relative to the system's support thickness.

4. SFC Turns Gap Bridging into a Governed Process

One of the most important consequences of SFC is that it gives Gap Bridging a clearer algorithmic structure.
Without SFC, gap bridging may be treated as a simple imperative:

- detect the gap,
- try to fill it.

But real systems need something more careful.
They need to ask:

- Is the gap inside confirmed territory, extension territory, or speculative territory?
- Does the gap have strong patchwork support?
- Is the missing piece peripheral or central?
- Is failure recoverable?
- Would a staged probe reduce uncertainty cheaply?
- Should another module or brain unit be called in?

This leads naturally to a more structured runtime procedure.

5. A Runtime Gap-Bridging Procedure Under SFC

A runtime informed by SFC may handle a gap through a sequence like the following.

Step 1 — Detect the Gap

Identify what is missing relative to the target:

- a missing task node,
- a missing action operator,
- a missing bridge edge,
- a missing decomposition step,
- or a missing state transition path.

Step 2 — Localize the Gap Relative to Known Support

Ask where the gap sits relative to:

- confirmed precedents,
- validated subgraphs,
- reusable modules,
- neighboring trajectories,
- and known decomposition patterns.

Step 3 — Estimate Structural Feasibility Confidence

Estimate whether the gap lies in:

- the Confirmed Zone,
- the Extension Zone,
- or the Speculative Zone.

Step 4 — Identify Patchwork Support

Check whether the gap is supported by:

- subtask coverage,
- module reuse,
- adjacent trajectories,
- mechanism understanding,
- or fallback strategies.

Step 5 — Select a Bridging Policy

Possible policies include:

- direct bridge,
- staged bridge,
- prototype first,
- ask for clarification,
- narrow the target,
- call another tool or brain unit,
- or defer.

Step 6 — Monitor Bridge Outcome and Update Support

If bridging succeeds, the system may update its tunnel thickness and future SFC for similar targets.
If bridging fails, it may revise the boundary, raise uncertainty, or reclassify that region as thinner than previously believed.
This is one of the clearest ways SFC becomes an algorithm rather than just a descriptive concept.

6. SFC and Runtime Commitment Governance

AI runtimes do not only need to execute.
They also need to govern their own promises.
A system interacting with users, tools, and other agents must decide:

- whether to accept a task,

- whether to claim confidence,
- whether to expose uncertainty,
- whether to promise an end-to-end answer,
- whether to propose a staged plan instead,
- and whether to say “I need another tool, another model, or more information.”

This is a commitment problem, not just an execution problem.
SFC therefore suggests that SCR should include some form of **commitment governance layer**—a layer that regulates what the system is willing to promise based on the thickness of structural support.
Without this layer, systems tend to oscillate between:

- overcommitting with shallow support, and
- refusing useful extension opportunities out of fear.

7. SFC and Task Acceptance

A future AI agent will often face a simple but crucial question:
“Should I take this task?”
That question should not be answered by a flat confidence score alone.
It should be answered by a structured feasibility analysis:

- How much of the task is already covered by confirmed structures?
- How much is extension work?
- Are the missing parts localized or systemic?
- Is there enough patchwork support to justify attempting it?
- Is the task better handled as one commitment or as multiple staged commitments?

This is especially important in multi-step agents, coding agents, research agents, and task orchestration systems.
Task acceptance is therefore one of the first practical domains where SFC

can become operational.

8. SFC and Multi-Brain / Multi-Tool Systems

SFC also has implications for multi-brain or multi-tool architectures.
Suppose a runtime detects that a target lies near or beyond the thickness boundary of its current active tunnel family.
That does not automatically imply refusal.
It may instead imply **escalation**.
Possible responses include:

- calling a specialized tool,
- invoking a domain-specific sub-agent,
- routing part of the task to a different brain unit,
- or decomposing the task into segments assigned to different capability clusters.

In this sense, SFC helps decide not only **whether to try**, but also **who should try** and **under what division of labor**.
This fits naturally with SCR and Brain Unit style architectures.

9. SFC and LLM/Transformer Scoring

SFC also helps reinterpret some current LLM behavior.
LLM scoring and ranking already contain a weak, implicit form of feasibility preference:
candidate continuations that fit contextual, semantic, and structural pattern support are scored more highly than candidates that do not.
But this implicit support field has major limitations:

- it is not explicitly typed into confirmed vs extension vs speculative zones,
- it does not naturally expose bridgeability,
- it often lacks causal mechanism awareness,
- and it does not by itself provide commitment governance.

SFC can therefore be viewed as a way of making implicit statistical support more explicit and more actionable.
Rather than merely preferring likely continuations, an SFC-aware runtime can

ask:

- *why* a continuation appears supported,
- *how strongly* it is supported,
- *which parts* of the task are thin,
- and *what policy* should follow from that support.

This is a major step from language continuation toward runtime intelligence.

10. SFC and Error Discipline

Another important implication is error discipline.
A runtime with no explicit SFC layer may fail in two opposite ways:

10.1 Unsupported Overextension

It assumes that because a target is linguistically plausible or superficially adjacent, it is safe to commit.

10.2 Frozen Conservatism

It assumes that because a target lacks exact precedent, it must not be attempted.
An SFC-aware runtime can do better by separating:

- unsupported speculation,
- bridgeable extension,
- and confirmed routine execution.

This allows the system to be both more adventurous and more disciplined at the same time.
That combination is exactly what good gap-bridging systems need.

11. SFC and Learning

SFC also suggests a natural learning loop for SCR.
Every time the runtime attempts a bridge, the result can update the support field.
If the bridge succeeds:

- a previously extension-only region may become partly confirmed,
- a new reusable module may be added,

- a new decomposition pattern may become available,
- and future confidence thickness may increase.

If the bridge fails:
- the runtime may discover hidden coupling,
- revise the estimated thickness boundary,
- add new caution markers,
- or learn that a particular extension direction is thinner than expected.

This means SFC is not static.
It can be continuously updated by runtime experience.
In that sense, SFC helps transform one-off gap bridging into cumulative capability growth.

12. SFC and User-Facing Interaction

For user-facing AI systems, SFC also has communication implications.
A system with explicit feasibility confidence can respond more honestly and more usefully:
- “I can do most of this, but the deployment section is outside my current support.”
- “I can draft the architecture now, but the performance estimate should be treated as provisional.”
- “I can attempt the bridge, but I recommend a probe step first.”
- “This task is feasible if we narrow the scope to these three components.”

This is better than both:
- fake certainty, and
- generic refusal.

It creates a path toward more transparent and cooperative AI behavior.

13. SFC as Extension Intelligence

Taken together, these implications suggest that SFC should be viewed as part of a broader runtime capability:

Extension Intelligence

Extension Intelligence is the ability to move from confirmed structure into adjacent novelty without collapsing into either rigid replay or unsupported speculation.

SFC contributes to that by supplying:

- a vocabulary for support,
- a geometry of confidence zones,
- a logic of patchwork proof,
- and a policy layer for commitment and gap bridging.

This makes it highly relevant to any SCR architecture that aims to be more than a replay engine.

14. Conclusion

Structural Feasibility Confidence has direct implications for SCR and AI agents because it governs one of the hardest practical questions in runtime intelligence:

When a target is not fully confirmed, how should the system decide whether to try, how strongly to commit, and how to manage the attempt?

That question appears in:

- gap bridging,
- task acceptance,
- staged planning,
- multi-tool routing,
- user-facing uncertainty communication,
- and cumulative capability growth.

SFC helps by providing a middle layer between structural observation and execution policy.

It tells the runtime not only that a gap exists, but whether the gap lies inside supported extension territory, what kind of bridge is justified, and when commitment should be conditional, staged, or refused.
In that sense, SFC is not merely an explanatory add-on to SCR.
It is a candidate component of runtime governance itself.

Key Takeaways

- SCR and future AI agents need an explicit way to judge **whether not-yet-confirmed targets are still within supported reach**.

- SFC provides a natural middle layer between:
  - asset/gap detection, and
  - execution/bridging/commitment policy.

- Under SFC, Gap Bridging becomes a governed process:
  - detect,
  - localize,
  - estimate support,
  - classify zone,
  - choose bridging policy,
  - update support after outcome.

- SFC naturally supports:
  - task acceptance,
  - staged commitment,
  - prototype-first strategies,
  - multi-tool / multi-brain escalation,
  - and better user-facing uncertainty communication.

- SFC can be seen as part of a broader **Extension Intelligence** layer inside

future runtime systems.